

Coding & Solution

Date	03 July 2026
Team ID	PNT2026TMID-EG01
Project Name	EduGenie - AI-Powered Educational Assistant
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/AP24110010223/EduGenie
Programming Language(s)	Python, JavaScript, HTML, CSS
Framework(s) Used	FastAPI, Uvicorn, Jinja2 (backend); vanilla JS, HTML5, CSS3 (frontend)
Key Features Implemented	AI-powered Question Answering using Gemini 1.5 Pro; Simplified Concept Explanation; AI Quiz Generation with topic-specific questions; Educational Text Summarization; Personalized Learning Roadmap recommendations; User authentication (register/login) with JWT; Learning history tracking; Interactive dashboard and profile management
Pending / Incomplete Features	Social login (Google/Facebook); Email confirmation on registration; Advanced analytics on the dashboard; Mobile-responsive polish for smaller screens
Setup / Run Instructions	1. Clone the repository: <code>git clone https://github.com/AP24110010223/EduGenie.git</code> 2. Backend: <code>cd backend</code> , create a virtual environment, run <code>pip install -r requirements.txt</code> 3. Add GEMINI_API_KEY and other secrets to a .env file 4. Start the backend: <code>uvicorn main:app --reload</code> 5. Frontend: open frontend/index.html via a local server (e.g., Live Server) or serve via FastAPI static files 6. Access the app at <code>http://127.0.0.1:8000</code>

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The repository follows a clean FastAPI project structure with clear separation between models, routers, schemas, and services. The `gemini_service.py` module centralizes all Generative AI calls, making the AI integration easy to maintain and extend. The frontend uses modular JS files (`api.js`, `auth.js`, `dashboard.js`) and shared CSS (`glass.css`, `style.css`) for consistent styling across all pages. The application runs locally without critical errors and is actively version controlled on GitHub.